

république Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique
Département SIQ

Mémoire de Master

Filière : Informatique
Spécialité : Réseau Et Systèmes Distribués

Thème:

**Conception et déploiement d'une infrastructure
Cloud privée hyperconvergée basée sur des
technologies open source**

Sujet Proposé par :

Mr : NEFFAH Mohamed

Soutenu le : /.././2026

Devant le jury composé de :

-
-

Présenté par :

- **Bouzara Zakaria**
- **Ouacherine Ilyes**

Binôme N° : 018RSD / 26

Année Universitaire : 2025/2026

I. SOMMAIRE

Introduction Générale.....	7
Chapitre 1 : Présentation du Contexte.....	9
1.1. Introduction.....	9
1.2. Présentation de Sonatrach.....	9
1.3.1. Infrastructure actuelle de Sonatrach.....	10
1.3.2. Les besoins en infrastructure légère pour les zones de test.....	11
1.4. Problématique.....	12
1.5. Objectifs du Projet.....	12
Objectif général.....	12
Objectifs spécifiques.....	13
1.6. Périmètre et Limitations du Projet.....	13
1.7. Conclusion.....	14
Chapitre 2 : État de l'Art et Concepts Théoriques.....	15
2.1. Introduction.....	15
2.2. L'Hyperconvergence (HCI).....	15
2.2.1. Définition.....	15
2.2.2. Avantages de l'hyperconvergence.....	16
2.2.3. HCI vs Infrastructure Traditionnelle.....	16
Tableau 2-1: comparaison du HCI avec Infrastructure Traditionnelle.....	16
2.3. Comparaison : Solutions Propriétaires vs Open Source.....	17
2.3.1. Solutions propriétaires.....	17
2.3.2. Solutions open source.....	17
2.4. Analyse des Solutions Open Source HCI.....	18
2.4.1. OpenStack.....	18
2.4.2. Harvester HCI.....	18
2.4.3. Proxmox VE.....	19
2.4.4. Tableau comparatif.....	20
Tableau 2-2: OpenStack vs Harvester vs Proxmox VE.....	20
2.5. Présentation des Technologies Retenues.....	20
2.5.1. Proxmox VE : L'hyperviseur.....	20
2.5.3. Open vSwitch (OVS) : La virtualisation réseau.....	25
2.6. L'Architecture "Inception" : La Virtualisation Imbriquée.....	27
2.7. Conclusion.....	29
Chapitre 3 : Conception de l'Architecture.....	30
3.1. Introduction.....	30

3.2. Vue Globale de l'Architecture.....	30
3.3. Architecture Physique et Virtualisation Imbriquée.....	31
3.3.1. Infrastructure Physique.....	31
3.3.2. Dimensionnement des Noeuds Virtuels.....	32
3.4. Architecture du Cluster Proxmox.....	34
3.4.1. Organisation du Cluster.....	34
3.4.2. Mécanisme de Quorum et Corosync.....	34
3.4.3. Haute Disponibilité.....	35
3.5. Architecture Réseau.....	36
3.5.1. Topologie Retenue.....	36
3.5.2. Plan d'Adressage.....	36
3.6. Architecture du Stockage Ceph.....	37
3.6.1. Dimensionnement du Pool.....	37
3.6.2. Réplication et Tolérance aux Pannes.....	38
3.7. Mécanisme d'Équilibrage Dynamique : DRS.....	39
3.7.1. Contexte et Besoin.....	39
3.7.2. Architecture du Mécanisme.....	39
3.7.3. Algorithme de Décision.....	40
3.7.4. Stack de Monitoring.....	42
3.8. Application de Démonstration : Sona-Web.....	42
3.8.1. Architecture Applicative.....	42
3.8.2. Scénario de Validation HA.....	43
3.9. Conclusion.....	43
Bibliographie.....	44

II. Table De Figures

Figure 3-1: Architecture Globale.....	31
Figure 3-2: Niveaux de Virtualisation Imbriquée.....	34
Figure 3-3: Topologie Réseau et Plan VLAN.....	37
Figure 3-4: Architecture du Mécanisme DRS.....	41
Figure 3-5: Architecture Applicative Sona-Web.....	45

III. Liste Des Tableaux

Tableau 2-1: comparaison du HCI avec Infrastructure Traditionnelle.....	16
Tableau 2-2: OpenStack vs Harvester vs Proxmox VE.....	20
Tableau 3-1 : Dimensionnement recommandé du serveur physique hôte.....	32
Tableau 3-2 : Allocation des ressources par noeud Proxmox virtuel.....	33
Tableau 3-3 : Plan d'adressage réseau et segmentation VLAN.....	37
Tableau 3-4 : Adressage IP statique des noeuds Proxmox par VLAN.....	37
Tableau 3-5 : Répartition du stockage Ceph par usage.....	38

IV. Liste Des Abréviations

Abréviation	Signification
BSD	Berkeley Software Distribution
CPU	Central Processing Unit
HA	High Availability
HCI	Hyper Converged Infrastructure
KVM	Kernel-based Virtual Machine
LXC	Linux Containers
MDS	MetaData Server
MGR	Manager
MON	Monitor
OSD	Object Storage Daemon
OVS	Open Virtual Switch
PC	Personal computer
PVE	Proxmox Virtual Environment
RAM	Random Access Memory
RBD	RADOS Block Device
RSD	Réseau Et Systèmes Distribués
TCO	Total Cost of Ownership
vCPU	Virtual Central Processing Unit
VE	Virtual Environment
VM	Virtual Machine

Introduction Générale

Ce mémoire est organisé en cinq chapitres :

- **Chapitre 1 - Introduction et Contexte** : présente le cadre général, la problématique et les objectifs du projet.
- **Chapitre 2 - Etat de l'Art** : analyse les solutions existantes (propriétaires et open source) et justifie le choix de la stack retenue.
- **Chapitre 3 - Conception de l'Architecture** : détaille l'architecture logique et physique : cluster Proxmox imbriqué, VLANs OVS, dimensionnement Ceph.
- **Chapitre 4 - Implémentation et Déploiement** : documente pas à pas l'installation, la configuration et le déploiement de l'application de démonstration.
- **Chapitre 5 - Validation et Tests** : présente les résultats du scénario de panne, les métriques de performance et les conclusions du projet.

Chapitre 1 : Présentation du Contexte

1.1. Introduction

Ce chapitre vise à présenter le contexte général dans lequel s'inscrit notre projet de fin d'études. Nous y aborderons les fondements de l'hyper convergence (HCI), les enjeux liés à la virtualisation et à la gestion unifiée des ressources informatiques, ainsi que les besoins spécifiques de l'entreprise Sonatrach. Ce cadre introductif nous permettra de mieux comprendre les choix technologiques effectués et les objectifs fixés pour le déploiement d'une infrastructure Cloud privée hyperconvergente basée sur des solutions open source.

1.2. Présentation de Sonatrach

Sonatrach est une entreprise publique algérienne créée le 31 décembre 1963, peu après l'indépendance de l'Algérie. Elle a été fondée dans le but de gérer, développer et exploiter les ressources hydrocarbures du pays, qui représentent une richesse stratégique pour l'économie nationale. À ses débuts, Sonatrach se consacrait essentiellement au transport et à la commercialisation du pétrole brut extrait en Algérie. [1]

Avec le temps, Sonatrach a évolué pour devenir la plus grande entreprise en Afrique dans le domaine des hydrocarbures. Elle intervient aujourd'hui dans l'ensemble de la chaîne de valeur pétrolière et gazière : exploration, production, transport, transformation et commercialisation. L'entreprise est également active dans la pétrochimie et les énergies renouvelables.

Sonatrach entretient de nombreuses collaborations internationales avec des compagnies pétrolières et gazières étrangères, impliquées dans de grands projets de partenariat via des joint-ventures et accords technologiques avec des entreprises européennes, américaines,

asiatiques et africaines. Ces coopérations ont permis un transfert de savoir-faire et de technologie vers l'Algérie.

Le projet est réalisé au sein du Centre d'innovation de Sonatrach. Cette direction joue un rôle stratégique dans l'innovation technologique et l'optimisation des processus internes. Elle dispose d'infrastructures informatiques avancées, notamment des solutions de virtualisation et de stockage haute capacité, destinées à soutenir les travaux de simulation, de test et de pré-production.

1.3. Infrastructure Existante et Contexte Technologique

1.3.1. Infrastructure actuelle de Sonatrach

Sonatrach dispose actuellement d'une infrastructure hyperconvergente basée sur la solution propriétaire Nutanix, déployée à l'échelle nationale sur plusieurs wilayas et utilisée pour les environnements de production critiques. Cette solution est reconnue pour sa stabilité, ses performances et la richesse de ses fonctionnalités, notamment en matière de gestion centralisée, de haute disponibilité et de scalabilité.

Cependant, cette infrastructure présente des contraintes structurelles qui deviennent de plus en plus préoccupantes dans une perspective d'évolution à long terme :

- **Coûts de licences croissants** : les frais de licence Nutanix suivent une trajectoire haussière, représentant une charge financière significative et récurrente pour l'entreprise, indépendamment du niveau d'utilisation réel de la plateforme.
- **Dépendance technologique forte** : le recours exclusif à un éditeur propriétaire unique expose Sonatrach à un risque de vendor lock-in, limitant sa capacité à négocier les conditions contractuelles et à adopter librement de nouvelles technologies.
- **Manque de souveraineté technique** : les évolutions de la plateforme, les mises à jour et les décisions d'architecture sont entièrement soumises à la roadmap de l'éditeur, sans possibilité d'adaptation aux besoins spécifiques de l'entreprise.

- **Inadaptation aux petits déploiements** : pour les environnements de test, de développement et de pré-production, le déploiement de Nutanix sur de petits clusters de 3 à 4 noeuds est disproportionné par rapport aux besoins réels, tant en complexité qu'en coût.

Ces constats positionnent ce projet non pas comme un simple exercice académique, mais comme une réponse concrète à une problématique stratégique réelle : explorer et valider la viabilité d'une infrastructure hyperconvergée open source capable de répondre aux exigences de Sonatrach, aujourd'hui en environnement de test et demain potentiellement en production.

1.3.2. Les besoins en infrastructure légère pour les zones de test

Au-delà des besoins immédiats en environnements de test et de pré-production, Sonatrach a besoin d'une infrastructure qui réponde à des exigences de niveau entreprise, tout en s'affranchissant des contraintes des solutions propriétaires. Ces besoins couvrent plusieurs dimensions :

- **Fiabilité et haute disponibilité** : l'infrastructure doit garantir la continuité de service en cas de défaillance d'un ou plusieurs noeuds, avec basculement automatique des charges de travail sans intervention humaine.
- **Performance et scalabilité** : la solution doit pouvoir monter en charge progressivement, en ajoutant des noeuds au cluster sans interruption de service, pour accompagner la croissance des besoins de l'entreprise.
- **Gestion unifiée des ressources** : calcul, stockage et réseau doivent être administrés depuis une interface centralisée unique, réduisant la complexité opérationnelle et les besoins en compétences spécialisées multiples.
- **Équilibrage dynamique de charge** : à l'image du mécanisme DRS de VMware, la plateforme doit être capable de redistribuer automatiquement les charges de travail entre les noeuds en fonction de l'utilisation des ressources, optimisant ainsi l'efficacité globale du cluster.

- **Souveraineté et indépendance technologique** : en s'appuyant exclusivement sur des technologies open source, l'entreprise reprend le contrôle total de son infrastructure, avec la liberté d'auditer, de modifier et d'adapter la solution à ses besoins propres.
- **Réduction du coût total de possession (TCO)** : l'absence de licences propriétaires représente une économie substantielle sur le long terme, réallouable vers des investissements matériels ou humains.

1.4. Problématique

La problématique centrale de ce projet est la suivante :

Comment concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement basée sur des technologies open source, capable de répondre aux exigences de fiabilité, de performance et de scalabilité d'une grande entreprise industrielle, tout en constituant une alternative crédible et souveraine aux solutions propriétaires de type Nutanix ?

Cette problématique soulève plusieurs questions techniques structurantes :

- Comment concevoir un système de stockage distribué, redondant et performant garantissant l'intégrité des données en cas de défaillance d'un nœud ?
- Comment assurer le basculement et l'équilibrage automatique des charges de travail dans un cluster à plusieurs nœuds, sans perte de données ni interruption prolongée de service ?
- Comment segmenter et isoler strictement les flux réseau au sein d'une infrastructure hyperconvergée, en séparant les trafics de gestion, de réplication, de migration et de production ?
- Dans quelle mesure une solution open source peut-elle répondre aux exigences d'une infrastructure de niveau production dans le contexte d'une grande entreprise ?

1.5. Objectifs du Projet

Objectif général

Concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement open source, offrant des garanties de fiabilité, de performance et d'évolutivité comparables aux solutions propriétaires de type Nutanix, et susceptible de constituer une alternative stratégique pour Sonatrach à moyen et long terme.

Objectifs spécifiques

- Déployer un cluster de trois nœuds à haute disponibilité, avec gestion centralisée, administration simplifiée et support de la migration à chaud.
- Concevoir et mettre en place un système de stockage distribué, redondant et performant intégrant les composants physiques et logiques nécessaires à la cohérence des données.
- Implémenter une segmentation réseau avancée, en configurant une architecture multi-VLAN séparant les flux de gestion, de réplication, de migration et de production.
- Développer un mécanisme d'équilibrage dynamique de la charge, capable de re-distribuer automatiquement les machines virtuelles selon les métriques de performance en temps réel.
- Valider la solution à travers un scénario de haute disponibilité démontrant la tolérance aux pannes sans perte de données ni interruption de service.
- Fournir une base d'infrastructure évolutive, accompagnée d'une documentation technique et d'une conception architecturale réutilisable à plus grande échelle.

1.6. Périmètre et Limitations du Projet

Ce projet vise le déploiement d'un cluster hyperconvergé open source destiné aux environnements de test, de développement et de pré-production. L'infrastructure est déployée sur un serveur dédié mis à disposition par Sonatrach, offrant les ressources matérielles nécessaires à l'exécution du cluster Proxmox VE, du stockage distribué Ceph et des VMs de démonstration.

Le périmètre de ce projet se limite volontairement à un environnement de laboratoire structuré, ce qui implique les contraintes suivantes :

- **Architecture sur serveur unique** : les trois nœuds Proxmox étant hébergés sur une même machine physique via virtualisation imbriquée, une panne matérielle totale du serveur hôte affecterait l'ensemble du cluster simultanément. Ce scénario ne se produirait pas dans un déploiement physique distribué sur plusieurs serveurs.
- **Absence de multitenancy** : Proxmox VE dans sa configuration actuelle est orienté cloud privé. La gestion multi-tenant propre aux clouds publics n'entre pas dans le périmètre de ce projet.
- **Capacité de stockage limitée** : la taille du pool Ceph est contrainte par l'espace disque disponible sur le serveur, ce qui rend cette infrastructure adaptée aux charges de test mais pas aux workloads de données à grande échelle.
- **Maintenance à long terme** : la gestion continue de l'infrastructure (mises à jour, sauvegardes automatisées, gestion des pannes matérielles) n'est pas couverte par ce projet et constitue une perspective d'évolution naturelle.

1.7. Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet, les besoins identifiés au sein de Sonatrach, et la problématique qui motive notre démarche. Nous avons défini les objectifs, le périmètre et les limites du projet. Ce cadre nous permettra d'aborder, dans le chapitre suivant, l'état de l'art des solutions disponibles et les fondements théoriques de l'hyperconvergence, de la virtualisation et des technologies retenues.

Chapitre 2 : État de l'Art et Concepts Théoriques

2.1. Introduction

Dans un contexte de transformation numérique accélérée, les organisations scientifiques et industrielles cherchent des solutions d'infrastructure à la fois performantes, flexibles et économiques. Ce chapitre présente les concepts théoriques fondamentaux sur lesquels repose notre projet : l'hyperconvergence, la virtualisation, le stockage distribué et la virtualisation du réseau. Il dresse également un état de l'art comparatif des solutions disponibles, afin de justifier les choix technologiques retenus.

2.2. L'Hyperconvergence (HCI)

2.2.1. Définition

L'hyperconvergence (HCI, Hyper-Converged Infrastructure) est une architecture informatique qui regroupe, dans un même système, les ressources de calcul, de stockage et de réseau. Ces ressources sont pilotées par une couche logicielle centralisée, supprimant ainsi la nécessité de gérer des équipements spécialisés distincts pour chaque fonction. [2]

L'infrastructure hyperconvergée repose sur trois composantes fondamentales :

- **Le calcul** : des processeurs virtuels (vCPU) alloués dynamiquement aux machines virtuelles.
- **Le stockage** : un système de stockage distribué logiciel, agrégé à partir des disques locaux de chaque nœud.
- **Le réseau** : un switch virtuel logiciel permettant la segmentation et l'isolation du trafic réseau.

2.2.2. Avantages de l'hyperconvergence

L'approche HCI présente plusieurs avantages par rapport aux architectures traditionnelles à trois niveaux (compute, storage, network séparés) :

- **Simplicité d'administration** : une interface unique pour gérer l'ensemble des ressources.
- **Réduction des coûts matériels** : pas besoin de baies de stockage ou de switchs réseau dédiés.
- **Évolutivité horizontale** : l'ajout d'un nœud augmente simultanément la capacité de calcul, de stockage et de réseau.
- **Haute disponibilité native** : la réplication des données entre les nœuds assure la continuité de service en cas de panne.

2.2.3. HCI vs Infrastructure Traditionnelle

Critère	Infrastructure Traditionnelle	Infrastructure HCI
Gestion	Silos séparés (compute, storage, network)	Interface unifiée
Coût initial	Elevé (équipements spécialisés)	Réduit (commodity hardware)
Évolutivité	Verticale (upgrade de matériel)	Horizontale (ajout de nœuds)
Complexité	Haute	Faible
Haute disponibilité	Optionnelle et complexe	Native

Tableau 2-1: comparaison du HCI avec Infrastructure Traditionnelle

2.3. Comparaison : Solutions Propriétaires vs Open Source

2.3.1. Solutions propriétaires

Les solutions propriétaires telles que **VMware vSAN** ou **Nutanix** offrent un support professionnel, des fonctionnalités avancées et une interface utilisateur soignée. Cependant, elles présentent plusieurs limites importantes dans notre contexte :

- **Coûts élevés** : les licences et la maintenance annuelle représentent un investissement significatif, difficilement justifiable pour des environnements de test.
- **Dépendance fournisseur (Vendor Lock-in)** : la migration vers une autre solution est souvent complexe et coûteuse.
- **Faible flexibilité** : les environnements propriétaires offrent peu de liberté pour personnaliser ou adapter l'infrastructure.

2.3.2. Solutions open source

Les solutions open source présentent plusieurs avantages stratégiques :

- **Indépendance technologique** : aucune dépendance vis-à-vis d'un éditeur.
- **Absence de coûts de licence** : seuls les coûts d'infrastructure matérielle et de main-d'œuvre sont à prévoir.
- **Transparence et auditabilité** : le code source est accessible et auditable.
- **Communauté active** : documentation abondante et mises à jour régulières.

Dans un contexte de zones de test et de petits déploiements, l'approche open source est clairement la plus adaptée.

2.4. Analyse des Solutions Open Source HCI

Le critère principal de sélection est la légèreté et la capacité à fonctionner de manière stable sur des clusters de 3 à 4 nœuds.

2.4.1. OpenStack

OpenStack est une plateforme cloud open source très complète et standardisée, utilisée par de nombreuses grandes entreprises et fournisseurs cloud. Elle propose une large gamme de services : calcul (Nova), stockage objet (Swift), stockage en bloc (Cinder), réseau (Neutron), identité (Keystone), etc.

Cependant, OpenStack présente des inconvénients majeurs pour notre cas d'usage :

- **Complexité de déploiement** : l'installation et la configuration d'OpenStack nécessitent une expertise poussée et prennent un temps considérable.
- **Consommation de ressources élevée** : les services de base d'OpenStack consomment à eux seuls plusieurs Go de RAM, avant même de démarrer des VMs de production.
- **Inadapté aux petits clusters** : OpenStack est conçu pour des déploiements de grande envergure. Sur 3 nœuds, ses performances et sa stabilité sont compromises.

2.4.2. Harvester HCI

Harvester est une solution HCI open source développée par SUSE, basée sur Kubernetes et KubeVirt. Elle propose une interface moderne et une intégration native avec Rancher pour la gestion de clusters Kubernetes. [5]

Ses limites dans notre contexte :

- **Consommation en RAM et CPU élevée** : Harvester nécessite des ressources importantes pour faire fonctionner la couche Kubernetes sous-jacente, avant même d'allouer des ressources aux VMs.

- **Maturité relative pour les VMs classiques** : Harvester est davantage orienté vers les workloads conteneurisés. La gestion de VMs Linux/Windows classiques est moins optimisée que sur une solution dédiée.
- **Courbe d'apprentissage** : la maîtrise de Kubernetes est un prérequis pour administrer efficacement Harvester. [4]

2.4.3. Proxmox VE

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur KVM (pour les VMs) et LXC (pour les conteneurs). Elle propose une interface web centralisée, un support natif de Ceph pour le stockage distribué, et une gestion intégrée du clustering et de la haute disponibilité. [6]

Ses atouts pour notre cas d'usage sont nombreux :

- **Légèreté** : moins de 1 Go de RAM consommé à vide sur le nœud, laissant l'essentiel des ressources aux VMs.
- **Déploiement rapide** : un cluster de 3 nœuds est opérationnel en quelques heures.
- **Interface web intuitive** : la console Proxmox (accessible sur le port 8006) centralise la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité.
- **Intégration native de Ceph** : Proxmox intègre nativement les outils de gestion Ceph, simplifiant considérablement le déploiement du stockage distribué.
- **Communauté active** : documentation officielle complète et forums très actifs.

2.4.4. Tableau comparatif

Critère	OpenStack	Harvester	Proxmox VE
Légèreté	Faible	Moyenne	Elevée
Facilité de déploiement	Difficile	Moyenne	Facile
Adapté aux petits clusters (3 noeuds)	Non	Partiel	Oui
Intégration stockage distribué	Via Cinder/Ceph	Via Longhorn	Ceph natif
Gestion des VMs classiques	Bonne	Moyenne	Excellente
Maturité	Très mature	En cours	Mature
Coût de licence	Gratuit	Gratuit	Gratuit (Enterprise payant)

Tableau 2-2: OpenStack vs Harvester vs Proxmox VE

Conclusion : Proxmox VE est la solution la mieux adaptée à nos contraintes. Elle sera retenue comme hyperviseur principal du projet.

2.5. Présentation des Technologies Retenues

2.5.1. Proxmox VE : L'hyperviseur

Proxmox VE est une solution de virtualisation complète basée sur un noyau Linux Debian modifié. Elle supporte deux types de virtualisation :

- **KVM (Kernel-based Virtual Machine)** : pour les machines virtuelles complètes, compatibles avec tout système d'exploitation (Linux, Windows, BSD...).
- **LXC (Linux Containers)** : pour les conteneurs Linux légers, partageant le noyau de l'hôte.

La gestion du cluster Proxmox repose sur **Corosync**, un framework de communication de cluster qui assure le consensus entre les nœuds (quorum). Un cluster Proxmox nécessite un minimum de 3 nœuds pour garantir un quorum stable et éviter les situations de “split-brain”.

2.5.2. Ceph : Le stockage distribué

Ceph est une solution libre de stockage distribuée (Software-Defined Storage) qui s'impose aujourd'hui comme une référence dans le domaine des infrastructures open source. Conçu pour répondre aux besoins de fiabilité, d'extensibilité et de performance, Ceph constitue le choix idéal pour assurer la couche de stockage d'une infrastructure hyperconvergée telle que celle que nous proposons. Sa capacité à fonctionner sur du matériel standard (serveurs x86) et son architecture auto-réparatrice en font un composant central de notre projet.

Architecture de Ceph

L'architecture de Ceph repose sur un composant fondamental appelé RADOS (Reliable Autonomic Distributed Object Store). RADOS constitue le cœur du système, chargé de la gestion, de la réplication et de la répartition des données sur l'ensemble du cluster. Autour de ce cœur gravitent plusieurs types de démons qui interagissent pour former un cluster cohérent et performant :

- Ceph Monitor (MON) : Les Moniteurs sont les gardiens du cluster. Ils maintiennent une carte maîtresse de l'état du cluster (cluster map), contenant la liste des nœuds, des OSD, et les règles de distribution des données. Pour assurer une haute disponibilité et éviter les conflits, un cluster doit comporter au minimum trois Moniteurs, en nombre impair, permettant une prise de décision par majorité en cas de panne.
- Ceph Manager (MGR) : Le Manager agit comme un chef d'orchestre. Il collecte toutes les métriques et informations d'état du cluster en temps réel (utilisation du stockage, performances, statistiques). Il expose ces données via une interface web de monitoring et les

met à disposition des systèmes de gestion externes, facilitant ainsi l'administration et la supervision.

- Ceph OSD (Object Storage Daemon) : L'OSD est le démon qui gère les périphériques de stockage physiques ou logiques (disques, partitions, volumes logiques). Chaque OSD est responsable de la lecture et de l'écriture des données sur son disque. Les OSDs communiquent entre eux pour assurer la réplication, le rééquilibrage et la réparation des données en cas de panne. Dans une architecture hyperconvergée, les OSDs résident sur les mêmes nœuds que l'hyperviseur (Proxmox), chaque nœud contribuant ainsi au stockage global.

- Ceph Metadata Server (MDS) : Ce démon est spécifique au système de fichiers distribué CephFS. Il gère les métadonnées des fichiers (noms, permissions, arborescences). Pour notre projet, qui utilise principalement le stockage en mode bloc pour les machines virtuelles, ce composant n'est pas requis.

La gestion intelligente des données : CRUSH et Placement Groups

L'intelligence de Ceph réside dans sa manière unique de gérer l'emplacement des données. Contrairement aux systèmes de stockage traditionnels qui utilisent une table de correspondance centralisée, Ceph utilise un algorithme appelé CRUSH (Controlled Replication Under Scalable Hashing). Cet algorithme permet à un client de calculer mathématiquement l'emplacement exact d'un objet (ou de sa réplique) en se basant sur la carte CRUSH, sans avoir à interroger un serveur central. Cette approche élimine un point de défaillance unique et améliore considérablement la scalabilité.

La carte CRUSH est une structure hiérarchique qui définit l'organisation physique du cluster. L'administrateur peut y définir des "domaines de défaillance" (nœuds, racks, salles, data centers). Ceph utilise ensuite ces informations pour placer les répliques des données sur des nœuds distincts, garantissant ainsi que la perte d'un équipement n'entraîne pas de perte de données.

Pour optimiser la gestion des milliers d'objets, Ceph les regroupe dans des unités logiques appelées Placement Groups (PGs) . Les PGs constituent une abstraction intermédiaire entre l'objet et l'OSD, permettant un équilibrage efficace des données et une meilleure performance du cluster.

Les modes de protection des données

Pour assurer la fiabilité et la disponibilité des données, Ceph propose deux mécanismes principaux, adaptés à différents besoins :

- Réplication : C'est le mode le plus simple et le plus performant. Chaque objet est dupliqué en N copies (généralement 3) et chaque copie est placée sur des OSDs différents, dans des domaines de défaillance distincts. En cas de panne d'un OSD ou d'un nœud, les données restent disponibles sur les autres répliques. Ce mode offre une reconstruction rapide et est idéal pour les charges de travail exigeant de hautes performances en écriture, comme les disques de machines virtuelles. Dans notre projet, nous utiliserons une réplication de facteur 3 pour garantir une haute disponibilité sur le cluster.
- Erasure Coding (Codage d'effacement) : Ce mode est plus efficace en termes d'espace de stockage. Les données sont découpées en K fragments de données, puis encodées avec M fragments de parité, pour un total de K+M fragments répartis sur différents OSDs. Ce mode permet de tolérer la perte de *M* fragments tout en reconstruisant les données initiales. Il est souvent utilisé pour les données froides ou l'archivage, car il nécessite moins d'espace que la réplication (ex: 1,5x l'espace brut contre 3x pour la réplication), mais il est plus coûteux en calcul lors des opérations de reconstruction.

Intégration de Ceph dans une architecture hyperconvergée

L'intégration de Ceph avec Proxmox VE, notre hyperviseur choisi, est native et profondément intégrée. Proxmox offre une interface unifiée qui permet de créer et de gérer un cluster Ceph directement depuis son interface web, simplifiant considérablement les opérations de déploiement et de maintenance.

Dans notre architecture, chaque nœud Proxmox joue un double rôle :

- Rôle de calcul : Il exécute les machines virtuelles via KVM.
- Rôle de stockage : Il contribue au stockage global via ses OSDs Ceph.

Cette convergence des rôles est l'essence même de l'hyperconvergence. Les volumes de stockage des VMs sont fournis via Ceph RBD (RADOS Block Device) , qui expose des périphériques de stockage en bloc sur le réseau. Ces volumes sont distribués et répliqués sur l'ensemble du cluster, assurant ainsi la persistance des données même en cas de panne d'un nœud.

Les avantages apportés par Ceph dans notre infrastructure sont multiples :

Caractéristique	Avantage pour l'infrastructure hyperconvergée
Évolutivité horizontale	L'ajout d'un nouveau nœud apporte simultanément de la puissance de calcul et de la capacité de stockage. Ceph rééquilibre automatiquement les données, permettant une croissance linéaire et sans interruption.
Haute disponibilité	L'absence de point de défaillance unique (SPOF) et la réplication des données sur plusieurs nœuds garantissent que le système de stockage reste opérationnel même en cas de panne d'un ou plusieurs serveurs.
Auto-réparation	En cas de panne d'un disque ou d'un nœud, le cluster détecte l'incident et commence immédiatement à recréer les répliques manquantes sur les autres nœuds, assurant ainsi la pérennité des données sans intervention manuelle.
Coût maîtrisé	Ceph est open source et fonctionne sur du matériel standard (x86), ce qui le rend bien moins coûteux que les solutions propriétaires comme les SAN ou les appliances HCI propriétaires.
Simplicité de gestion	L'unification du stockage via Proxmox et les outils modernes comme 'cephadm' (permettant un déploiement conteneurisé et simplifié) réduisent considérablement la complexité administrative.

Conclusion

Ceph représente bien plus qu'un simple système de stockage. Son architecture robuste, sa scalabilité horizontale et sa capacité d'auto-guérison s'alignent parfaitement avec les principes

d'une infrastructure hyperconvergée. En choisissant Ceph comme brique de stockage, nous posons des fondations solides pour une infrastructure IT moderne, agile et résiliente, capable de s'adapter aux besoins futurs. L'intégration native avec Proxmox VE permet une gestion centralisée et simplifiée, essentielle pour les environnements de test et de pré-production visés par ce projet.

2.5.3. Open vSwitch (OVS) : La virtualisation réseau

Open vSwitch (OVS) est un commutateur logiciel multicouche open source, distribué sous licence Apache 2.0, dont l'objectif est de fournir une plateforme de commutation de qualité production supportant les interfaces de gestion standard et ouvrant les fonctions de transfert à l'extension programmatique [8]. Particulièrement adapté aux environnements de virtualisation, OVS est conçu pour être distribué sur plusieurs serveurs physiques et supporte les technologies de virtualisation Linux dont KVM et VirtualBox [8]. Il fonctionne principalement en deux composants : **ovs-vsitchd**, le démon qui implémente le switch avec son module noyau Linux pour un transfert haute performance, et **ovsdb-server**, une base de données légère de configuration [8]. Parmi ses fonctionnalités natives figurent le modèle VLAN 802.1Q, le bonding avec ou sans LACP, la supervision via NetFlow et sFlow, la configuration QoS, ainsi que le support des tunnels GRE, VXLAN et Geneve [9].

Dans notre infrastructure, OVS joue le rôle de commutateur central du cluster, assurant la segmentation de tout le trafic en flux distincts et isolés. Cette segmentation est imposée par la coexistence sur la même infrastructure physique de trafics aux profils radicalement différents : réplication Ceph à très haute bande passante, heartbeats Corosync sensibles à la latence, pics de migration à chaud, et workloads applicatifs variables. Sans isolation stricte, ces flux en compétition dégradent mutuellement leurs performances et menacent la stabilité du cluster. OVS résout ce problème depuis un bridge unique par noeud, à l'image de la séparation des réseaux CVM pratiquée par Nutanix.

La conception de notre architecture réseau a été guidée par plusieurs facteurs déterminants.

- Le premier est la nature hétérogène des flux coexistant dans un cluster HCI : chaque type de trafic présente un profil distinct en termes de bande passante, de sensibilité à la latence et de criticité opérationnelle, rendant leur cohabitation sur un réseau commun incompatible avec les exigences de performance et de stabilité d'une infrastructure de niveau entreprise.

- Le deuxième facteur est la référence aux recommandations officielles de la documentation Proxmox et aux guidelines Ceph, qui préconisent explicitement la séparation des réseaux public et cluster Ceph [10] [12], ainsi que l'isolation du trafic Corosync [11].

- Le troisième facteur est le positionnement du projet comme alternative à Nutanix : nous avons analysé la manière dont Nutanix segmente ses propres réseaux internes entre trafic CVM, trafic de réplication et trafic VM, et avons traduit cette logique en VLANs OVS équivalents.

- Enfin, la nécessité d'anticiper la scalabilité du cluster nous a conduits à prévoir dès la conception des segments dédiés à des fonctions comme la migration automatique et la sauvegarde, que le mécanisme DRS que nous développons sollicitera de manière intensive.

Ces paramètres ont abouti à une architecture à neuf VLANs distincts, chacun correspondant à un type de trafic précis avec une plage d'adressage dédiée :

- **VLAN 10 - Management (192.168.10.0/24)** : accès à l'interface web Proxmox, SSH et API. Faible volume mais disponibilité permanente requise.
- **VLAN 20 - Corosync (192.168.20.0/24)** : heartbeats et quorum du cluster.
Extrêmement sensible à la latence : toute perturbation peut déclencher un basculement non souhaité.
- **VLAN 30 - Ceph Cluster Network (10.10.30.0/24)** : communications internes entre OSDs, réplication, rebalancing et recovery. Trafic continu à très haute bande passante, isolé conformément aux recommandations officielles Ceph. [10] [12] [13]

- **VLAN 40 - Ceph Public Network (10.10.40.0/24)** : accès des VMs au stockage Ceph via RBD. Séparé du VLAN 30 pour éviter que les opérations internes Ceph ne dégradent les performances applicatives. [12] [13]
- **VLAN 50 - Live Migration (10.10.50.0/24)** : migrations à chaud entre noeuds. Trafic soutenu et ponctuel, particulièrement sollicité par le mécanisme DRS que nous développons.
- **VLAN 60 - VM Production (172.16.60.0/24)** : workloads applicatifs de production, dont Sona-Web. Point d'entrée utilisateur via HAProxy.
- **VLAN 70 - VM Test/Dev (172.16.70.0/24)** : environnements de développement et validation, strictement isolés de la production.
- **VLAN 80 - DMZ / External (172.16.80.0/24)** : services exposés vers l'extérieur de l'entreprise, protégés par pare-feu. HAProxy est le seul point d'entrée vers cette zone.
- **VLAN 90 - Backup (10.10.90.0/24)** : transferts de sauvegarde et snapshots Proxmox, isolés pour ne pas impacter les workloads en production.

2.6. L'Architecture "Inception" : La Virtualisation

Imbriquée

Notre projet adopte une architecture de virtualisation imbriquée (nested virtualization), parfois appelée architecture "Inception". Cette approche consiste à faire fonctionner un hyperviseur à l'intérieur d'un autre hyperviseur, permettant de déployer un cluster multi-noeuds complet sur une seule infrastructure physique dédiée.

Le principe est organisé en trois niveaux distincts :

- **Niveau 0 (Physique) :** le serveur dédié mis à disposition par Sonatrach héberge Proxmox VE directement en bare metal, jouant le rôle d'hyperviseur hôte. C'est la fondation physique sur laquelle repose l'intégralité de l'infrastructure.
- **Niveau 1 (Noeuds virtuels) :** à l'intérieur de cet hyperviseur hôte, trois VMs de grande capacité sont créées et configurées pour jouer le rôle des noeuds Proxmox du cluster (PVE-01, PVE-02, PVE-03). Chacune de ces VMs exécute sa propre instance Proxmox VE, ses daemons Ceph (MON, OSD, MGR) et son instance OVS. Ces trois noeuds forment un cluster Proxmox complet et fonctionnel.
- **Niveau 2 (VMs applicatives) :** le cluster Proxmox de niveau 1 héberge à son tour les VMs de démonstration constituant l'application Sona-Web : le load balancer HAProxy, les serveurs web Apache et la base de données PostgreSQL.

Cette architecture présente plusieurs avantages qui justifient son adoption dans le cadre de ce projet :

Isolation complète : l'ensemble du cluster est encapsulé dans le serveur hôte, garantissant une isolation totale vis-à-vis du reste du réseau Sonatrach pendant la phase de développement et de test.

Réseau 100% virtuel et performant : toutes les communications inter-noeuds, y compris le trafic Ceph et Corosync, transitent par le bus mémoire interne du serveur hôte. Cela élimine toute dépendance à un switch physique externe et garantit une latence minimale et une bande passante maximale entre les noeuds, condition particulièrement favorable pour les performances du cluster Ceph.

Snapshots du cluster complet : il est possible de prendre un snapshot de l'intégralité du cluster au niveau de l'hyperviseur hôte avant chaque démonstration ou opération critique, permettant un retour à un état stable en quelques secondes si nécessaire.

Portabilité et reproductibilité : l'ensemble de l'environnement peut être exporté, sauvegardé ou reproduit sur un autre serveur de caractéristiques similaires, facilitant la transmission et la réutilisation de l'infrastructure dans d'autres contextes au sein de Sonatrach.

Évolutivité vers la production : lorsque l'infrastructure sera amenée à évoluer vers un déploiement distribué sur plusieurs serveurs physiques, la migration sera transparente. Les trois noeuds virtuels pourront être convertis en noeuds physiques indépendants sans modification de la configuration logicielle du cluster, garantissant ainsi la continuité et la réutilisabilité du travail réalisé.

2.7. Conclusion

Ce chapitre a permis de poser les bases théoriques de notre projet. Nous avons défini le concept d'hyperconvergence, comparé les principales solutions open source disponibles et justifié le choix de la stack Proxmox VE + Ceph + Open vSwitch. Nous avons également présenté le rôle de chaque composant dans l'architecture et introduit le concept de virtualisation imbriquée qui sera utilisé dans la phase d'implémentation. Le chapitre suivant présentera la conception détaillée de l'architecture proposée.

Chapitre 3 : Conception de l'Architecture

3.1. Introduction

Ce chapitre présente la conception détaillée de l'infrastructure Cloud privée hyperconvergée que nous proposons. Sur la base des technologies retenues dans le chapitre précédent, Proxmox VE, Ceph et Open vSwitch, nous décrivons ici les choix architecturaux qui ont guidé la construction de chaque couche de l'infrastructure : physique, réseau, stockage et applicative. Nous présentons également la conception du mécanisme d'équilibrage dynamique de charge que nous développons en complément de la stack de base.

L'objectif de ce chapitre est de fournir une vision complète et cohérente de l'architecture avant toute phase d'implémentation, en justifiant chaque décision de conception par des contraintes techniques, des recommandations officielles ou des exigences fonctionnelles identifiées dans le chapitre précédent.

3.2. Vue Globale de l'Architecture

L'architecture proposée repose sur un modèle à trois couches superposées et interdépendantes, chacune apportant une dimension essentielle à l'infrastructure hyperconvergée.

La première couche est la **couche physique**, constituée du serveur dédié mis à disposition par Sonatrach. Ce serveur héberge Proxmox VE directement en bare metal et constitue la fondation matérielle sur laquelle repose l'intégralité de l'infrastructure. Selon les recommandations officielles de la documentation Proxmox [10], un serveur destiné à héberger un cluster Ceph doit disposer de ressources suffisantes pour absorber simultanément la charge des hyperviseurs, des daemons de stockage et des workloads applicatifs.

La deuxième couche est la **couche de virtualisation et de cluster**, composée de trois noeuds Proxmox virtuels (PVE-01, PVE-02, PVE-03) déployés en virtualisation imbriquée sur le serveur hôte. Ces trois nœuds forment un cluster Proxmox haute disponibilité géré par Corosync pour le consensus et la détection des pannes. Chaque noeud exécute simultanément les daemons Ceph nécessaires au stockage distribué et une instance Open vSwitch pour la gestion du trafic réseau. Cette convergence des rôles au sein de chaque noeud est l'expression directe du principe HCI.

La troisième couche est la **couche applicative et de services**, hébergée par le cluster Proxmox. Elle comprend les VMs de démonstration constituant l'application Sona-Web, ainsi que les services d'infrastructure transversaux : le mécanisme DRS que nous développons, la stack de monitoring InfluxDB et Grafana, et le serveur de sauvegarde Proxmox Backup Server.

Ces trois couches interagissent de manière unifiée depuis l'interface d'administration centralisée Proxmox, accessible sur le VLAN 10 de management. La figure 3-1 illustre cette organisation globale.

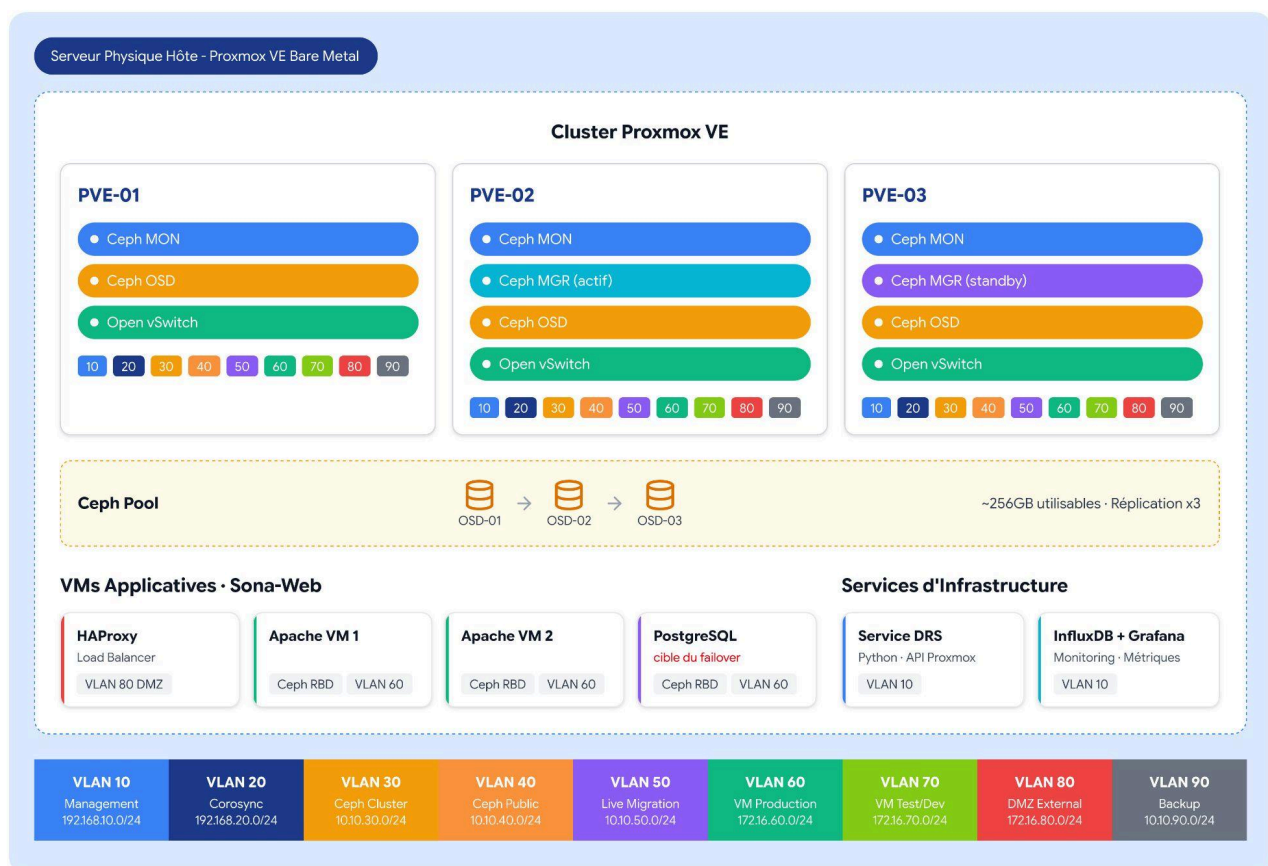


Figure 3-1: Architecture Globale

3.3. Architecture Physique et Virtualisation Imbriquée

3.3.1. Infrastructure Physique

Le serveur physique constitue le Niveau 0 de l'architecture. Il héberge Proxmox VE en bare metal sans aucune couche de virtualisation intermédiaire, garantissant des performances optimales pour l'hyperviseur hôte. Le système d'exploitation Proxmox VE est basé sur Debian Linux avec un noyau modifié intégrant les optimisations KVM nécessaires à la virtualisation imbriquée [6].

Pour un déploiement de ce type, la documentation Proxmox recommande une séparation physique entre le disque système de l'hôte et les disques destinés aux OSDs Ceph [10]. Cette séparation évite toute compétition d'I/O entre le système hôte et le stockage distribué, ce qui aurait un impact direct sur les performances du cluster Ceph. Les recommandations Ceph précisent par ailleurs que les disques OSD bénéficient significativement de l'utilisation de SSDs ou NVMe, particulièrement pour les dispositifs WAL (Write-Ahead Log) et DB (RocksDB metadata) qui absorbent les pics d'écriture [13].

Le tableau suivant présente le dimensionnement recommandé pour le serveur hôte, établi sur la base des prérequis officiels Proxmox [10], des recommandations Ceph [12][13] et des travaux de Chandak et al. sur l'allocation des ressources dans les environnements KVM multi-noeuds [14] :

Composant	Valeur recommandée	Justification
CPU	16 cores physiques minimum	3 noeuds $\times$ 4 vCPUs + overhead hôte [10]
RAM	96 GB minimum	3 noeuds $\times$ 24 GB + 8 GB hôte + marge [10][14]
Disque système hôte	SSD 120 GB dédié	Isolation I/O hôte / Ceph [10]
Disques OSD	3 $\times$ SSD 256 GB minimum	Un OSD par noeud virtuel [12][13]
Disque WAL/DB Ceph	SSD NVMe 64 GB dédié	Performances d'écriture Ceph [13]
NICs	2 $\times$ 1 GbE minimum, 10 GbE recommandé	Bonding LACP, trafic OVS [11]

Tableau 3-1 : Dimensionnement recommandé du serveur physique hôte

Ces valeurs seront ajustées en fonction des spécifications exactes du serveur mis à disposition par Sonatrach.

3.3.2. Dimensionnement des Noeuds Virtuels

Chaque nœud virtuel (PVE-01, PVE-02, PVE-03) reçoit une allocation de ressources identique afin de garantir l'équilibre du cluster et de simplifier l'administration. Cette symétrie est une bonne pratique recommandée par la documentation Proxmox pour les clusters Ceph [10], car un déséquilibre de ressources entre noeuds peut conduire Ceph à surcharger certains OSDs au détriment d'autres.

Chaque noeud virtuel est configuré avec deux disques virtuels distincts, conformément aux recommandations Proxmox [10] : un premier disque pour le système Proxmox du noeud, et un second disque non formaté, donné brut à Ceph pour constituer un OSD. Cette séparation est fondamentale car elle évite que les opérations de réplication Ceph ne concurrencent les I/O système du noeud.

Composant	Valeur par noeud	Justification
vCPUs	6	Proxmox + Ceph OSD + OVS + VMs [10][14]
RAM	24 GB	6 GB Ceph + 4 GB Proxmox + 14 GB VMs [14]
Disque OS noeud	32 GB virtio	Système Proxmox du noeud
Disque OSD	256 GB virtio, non formaté	OSD Ceph, donné brut au daemon [12]
Interfaces réseau	9 interfaces OVS virtuelles	Une par VLAN selon notre architecture réseau
Type de virtualisation	KVM avec nested virt activé	Support Intel VT-x / AMD-V imbriqué [6]

Tableau 3-2 : Allocation des ressources par noeud Proxmox virtual

La virtualisation imbriquée nécessite l'activation explicite du flag CPU host ou kvm64 sur le serveur hôte, permettant aux noeuds virtuels d'accéder aux extensions de virtualisation

matérielle du processeur physique. Sans cette configuration, les performances des VMs de niveau 2 seraient significativement dégradées [6].

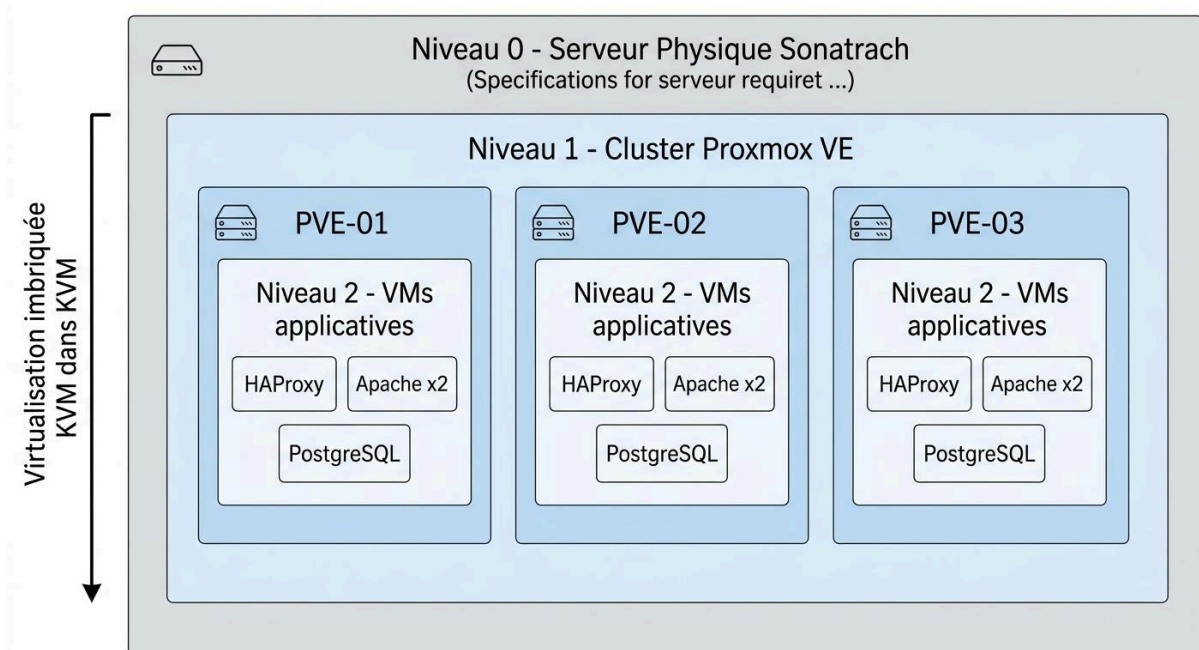


Figure 3-2: Niveaux de Virtualisation Imbriquée

3.4. Architecture du Cluster Proxmox

3.4.1. Organisation du Cluster

Le cluster Proxmox est constitué de trois nœuds virtuels identiques, interconnectés via le réseau Corosync dédié (VLAN 20). Cette organisation à trois nœuds est le minimum requis par Proxmox pour garantir un quorum stable et éviter les situations de split-brain [21]. Un cluster à deux nœuds ne peut pas déterminer par lui-même lequel des deux nœuds est défaillant en cas de perte de communication, ce qui rendrait le système incapable de prendre des décisions automatiques de basculement.

Chaque nœud du cluster joue simultanément plusieurs rôles, conformément au principe HCI :

- **Rôle hyperviseur** : exécution des VMs applicatives via KVM
- **Rôle stockage** : participation au cluster Ceph via ses daemons MON, OSD et MGR
- **Rôle réseau** : gestion du trafic via Open vSwitch

Cette polyvalence est précisément ce qui distingue une architecture HCI d'une architecture traditionnelle à trois tiers séparés.

3.4.2. Mécanisme de Quorum et Corosync

Corosync est le framework de communication de cluster utilisé par Proxmox pour assurer le consensus entre les nœuds. Il envoie des messages de heartbeat entre les nœuds à intervalles réguliers via le VLAN 20 dédié. Lorsqu'un nœud cesse de répondre, Corosync déclenche une procédure de vote pour déterminer si le cluster dispose encore du quorum nécessaire pour continuer à opérer [20].

Le quorum dans un cluster de trois nœuds est atteint dès que deux nœuds sur trois sont opérationnels. Cela signifie que le cluster peut tolérer la perte d'un nœud complet tout en continuant à fonctionner normalement, ce qui satisfait l'exigence de haute disponibilité du projet. La perte de deux nœuds simultanément entraîne la perte du quorum et le gel du cluster pour éviter toute corruption des données, comportement conforme aux recommandations de sécurité Proxmox [6].

L'isolation du trafic Corosync sur le VLAN 20 dédié est essentielle. Des études sur la stabilité des clusters KVM montrent que la compétition réseau entre le trafic de heartbeat et d'autres flux à haute bande passante est l'une des causes les plus fréquentes de faux positifs de panne dans les environnements virtualisés [14]. Notre architecture prévient ce risque en garantissant que le VLAN 20 n'est jamais partagé avec d'autres types de trafic.

3.4.3. Haute Disponibilité

Le mécanisme de haute disponibilité (HA) de Proxmox s'appuie sur Corosync pour la détection des pannes et sur le Resource Manager HA pour les décisions de basculement [6].

Lorsqu'un noeud est déclaré défaillant par le quorum Corosync, le HA Manager identifie les VMs qui y résidaient et déclenche leur redémarrage automatique sur un des noeuds survivants.

Ce mécanisme est rendu possible par le stockage partagé Ceph : les disques des VMs étant stockés sur le pool Ceph commun et accessibles depuis n'importe quel noeud du cluster, le redémarrage d'une VM sur un noeud différent ne nécessite aucun transfert de données. La VM redémarre directement depuis le volume Ceph RBD existant, ce qui minimise le temps de récupération après panne (RTO). Selon la documentation Proxmox, ce délai de basculement est typiquement compris entre 60 et 120 secondes selon la configuration [6][22].

Dans notre architecture, toutes les VMs de l'application Sona-Web sont configurées en mode HA, avec une priorité de basculement définie pour garantir que le load balancer HAProxy est redémarré en premier, suivi des serveurs web, puis de la base de données PostgreSQL.

3.5. Architecture Réseau

3.5.1. Topologie Retenue

Comme établi dans le chapitre 2, nous adoptons une topologie hybride combinant Full Mesh pour le trafic Ceph et Switched VLANs sur OVS pour l'ensemble des autres flux. Cette topologie est celle recommandée par la documentation officielle Proxmox pour les clusters de 3 à 5 noeuds [11].

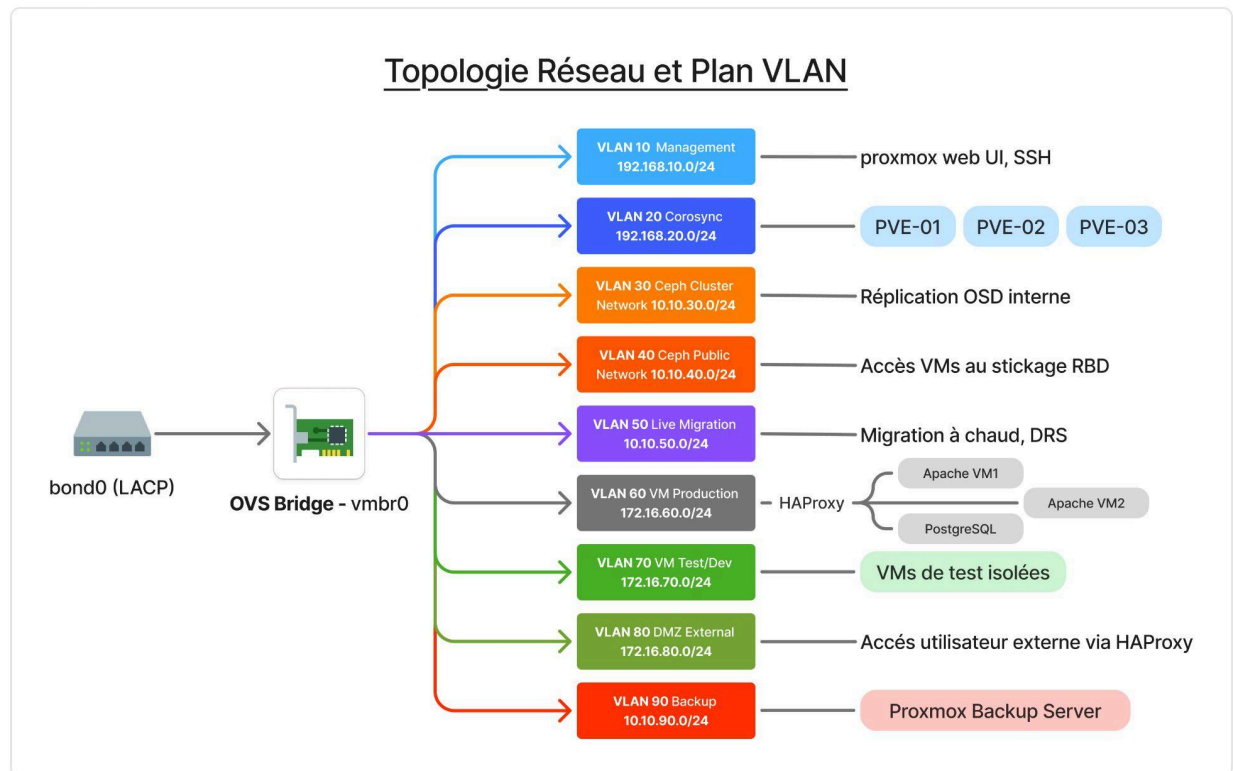


Figure 3-3: Topologie Réseau et Plan VLAN

Dans le contexte de la virtualisation imbriquée, l'ensemble des liens réseau inter-noeuds sont des interfaces virtuelles internes au serveur hôte. Cela signifie que le Full Mesh Ceph est implémenté via des interfaces virtuelles directes entre les noeuds de niveau 1, sans passer par un switch physique externe. Les performances réseau sont ainsi limitées uniquement par la bande passante du bus mémoire du serveur hôte, ce qui constitue un avantage significatif par rapport à un réseau physique 1 GbE.

3.5.2. Plan d'Adressage

Le plan d'adressage complet de l'infrastructure est le suivant :

VLAN	Nom	Subnet	Passerelle	Usage
10	Management	192.168.10.0/24	192.168.10.1	Web UI, SSH, API Proxmox
20	Corosync	192.168.20.0/24	192.168.20.1	Heartbeats, quorum cluster
30	Ceph Cluster	10.10.30.0/24	10.10.30.1	Réplication OSD interne
40	Ceph Public	10.10.40.0/24	10.10.40.1	Accès VMs au stockage RBD
50	Live Migration	10.10.50.0/24	10.10.50.1	Migrations à chaud, DRS
60	VM Production	172.16.60.0/24	172.16.60.1	Sona-Web production
70	VM Test/Dev	172.16.70.0/24	172.16.70.1	Environnements de test
80	DMZ/External	172.16.80.0/24	172.16.80.1	Services exposés, HAProxy
90	Backup	10.10.90.0/24	10.10.90.1	Sauvegardes PBS

Tableau 3-3 : Plan d'adressage réseau et segmentation VLAN

Pour chaque noeud Proxmox virtuel, les adresses IP sont attribuées de manière statique selon le schéma suivant, où X représente le numéro du noeud (1, 2 ou 3) :

Noeud	VLAN 10	VLAN 20	VLAN 30	VLAN 40
PVE-01	192.168.10.11	192.168.20.11	10.10.30.11	10.10.40.11
PVE-02	192.168.10.12	192.168.20.12	10.10.30.12	10.10.40.12
PVE-03	192.168.10.13	192.168.20.13	10.10.30.13	10.10.40.13

Tableau 3-4 : Adressage IP statique des noeuds Proxmox par VLAN

3.6. Architecture du Stockage Ceph

3.6.1. Dimensionnement du Pool

Le cluster Ceph est constitué de trois OSDs, un par noeud virtuel, chacun disposant de 256 GB de stockage brut. Avec un facteur de réplication de 3, recommandé par la documentation Ceph pour les environnements nécessitant une haute disponibilité [12][13], la capacité utilisable du pool est calculée comme suit :

Capacité utilisable = Capacité brute totale / Facteur de réplication
Capacité utilisable = $(3 \times 256 \text{ GB}) / 3 = 256 \text{ GB}$

Cette capacité est répartie entre les VMs applicatives et les services d'infrastructure selon le tableau suivant :

Usage	Stockage alloué
Disques OS des 3 noeuds Proxmox	$3 \times 32 \text{ GB} = 96 \text{ GB}$
HAProxy VM	16 GB
Apache VM 1	32 GB
Apache VM 2	32 GB
PostgreSQL VM	100 GB
DRS Service VM	32 GB
InfluxDB + Grafana VM	100 GB
Marge et snapshots	~50 GB
Total estimé	~458 GB

Tableau 3-5 : Répartition du stockage Ceph par usage

Ce dimensionnement suppose un disque OSD de 256 GB par noeud comme valeur minimale recommandée. Des disques OSD plus grands permettraient d'accroître la capacité utilisable proportionnellement.

3.6.2. Réplication et Tolérance aux Pannes

La réplication de facteur 3 garantit que chaque bloc de données est écrit simultanément sur trois OSDs distincts, résidant sur trois noeuds différents. En cas de perte d'un noeud complet, les deux copies restantes assurent la disponibilité continue des données sans interruption de service ni perte de données [12][13].

L'algorithme CRUSH de Ceph est configuré pour respecter les domaines de défaillance au niveau du noeud, garantissant que les trois répliques d'un même bloc ne se retrouvent jamais sur le même noeud physique virtuel [12]. Dans notre architecture à trois noeuds, cela signifie que chaque réplique réside sur un noeud différent, offrant la tolérance maximale possible avec ce nombre de noeuds.

3.7. Mécanisme d'Équilibrage Dynamique : DRS

3.7.1. Contexte et Besoin

Proxmox VE intègre nativement un mécanisme de haute disponibilité réactif : lorsqu'un noeud tombe en panne, les VMs qui y résidaient sont automatiquement redémarrées sur les noeuds survivants. Cependant, Proxmox ne dispose d'aucun mécanisme proactif d'équilibrage de charge, contrairement à VMware vSphere qui propose le DRS (Distributed Resource Scheduler) [18].

Sans équilibrage proactif, un cluster Proxmox peut se retrouver dans une situation où un noeud est saturé à 90% de CPU tandis qu'un autre noeud reste à 20% d'utilisation. Proxmox n'intervient pas dans ce cas, laissant le déséquilibre persister et dégradant les performances des VMs sur le noeud surchargé. À l'échelle d'un déploiement entreprise comme Sonatrach,

ce déséquilibre représente un gaspillage significatif de ressources et une dégradation de la qualité de service [18].

Le travail de Watts (1995) sur les approches pratiques de l'équilibrage dynamique de charge établit que l'efficacité d'un mécanisme de load balancing repose sur trois piliers fondamentaux : la collecte précise des métriques de charge, un algorithme de décision évitant les oscillations, et un mécanisme de migration à faible coût [17]. Notre architecture répond à ces trois exigences respectivement via InfluxDB, un algorithme à seuils avec fenêtre glissante, et la live migration Proxmox via le VLAN 50.

3.7.2. Architecture du Mécanisme

Le mécanisme DRS que nous développons est un service externe à Proxmox, interagissant avec l'infrastructure via deux interfaces : l'API REST Proxmox pour les actions de migration, et InfluxDB pour la collecte des métriques historiques. Cette approche non intrusive est délibérée : elle ne modifie pas le comportement interne de Proxmox et reste compatible avec toutes les futures versions de la plateforme.

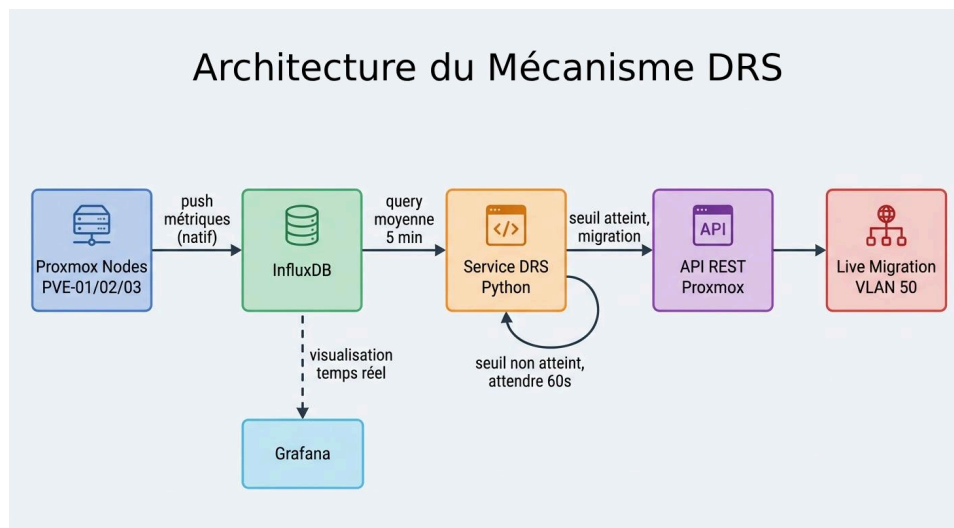


Figure 3-4: Architecture du Mécanisme DRS

Le mécanisme s'articule autour de quatre composants principaux, dont l'interaction est illustrée dans la figure 3-2 :

- **Proxmox Metrics Exporter** : Proxmox VE supporte nativement l'export de métriques vers InfluxDB via son interface de configuration [15]. Chaque noeud pousse automatiquement ses métriques de CPU, RAM, réseau et stockage vers la base de données InfluxDB à intervalles réguliers, sans nécessiter d'agent supplémentaire sur les noeuds.
- **InfluxDB** : base de données de séries temporelles qui stocke l'historique des métriques de tous les noeuds et VMs du cluster. L'utilisation d'une base de données temporelle plutôt que de requêtes instantanées à l'API Proxmox est fondamentale pour la stabilité du mécanisme : les décisions de migration sont prises sur des moyennes glissantes sur 5 minutes, évitant ainsi les migrations déclenchées par des pics de charge ponctuels et éphémères [24].
- **Service DRS (Python)** : le coeur du mécanisme. Ce service s'exécute en continu sur une VM dédiée et implémente la boucle de décision principale. Il interroge InfluxDB pour obtenir les métriques moyennées, applique l'algorithme de décision, et déclenche les migrations via l'API REST Proxmox lorsque les conditions sont réunies.
- **Grafana** : tableau de bord de visualisation connecté à InfluxDB. Il affiche en temps réel l'utilisation des ressources par noeud, l'historique des migrations déclenchées par le DRS, et les alertes de surcharge. Ce composant est particulièrement utile lors de la démonstration pour rendre visible le comportement du mécanisme DRS au jury [25].

3.7.3. Algorithme de Décision

L'algorithme de décision du DRS repose sur un système de seuils à deux niveaux, inspiré des travaux de Watts sur l'équilibrage dynamique [17] et du comportement documenté de VMware DRS [18] :

- **Seuil de surcharge (HIGH_THRESHOLD)** : fixé à 80% d'utilisation CPU ou 85% d'utilisation RAM sur un nœud, calculé comme moyenne sur une fenêtre glissante de 5 minutes. Au-delà de ce seuil, le nœud est considéré comme surchargé et le DRS cherche une VM candidate à migrer.

- **Seuil de sous-charge (LOW_THRESHOLD)** : fixé à 30% d'utilisation CPU et RAM. Un nœud sous ce seuil est considéré comme candidat à recevoir une VM migrée.

- **Critère de sélection de la VM à migrer** : parmi toutes les VMs du nœud surchargé, le DRS sélectionne celle dont le rapport charge/taille mémoire est le plus favorable à la migration, c'est-à-dire la VM la plus lourde en consommation CPU mais dont la mémoire est suffisamment réduite pour minimiser le temps de migration sur le VLAN 50.

- **Garde-fous anti-oscillation** : une VM migrée ne peut pas être migrée à nouveau avant un délai de 10 minutes, évitant le phénomène de "ping-pong" décrit par Watts comme l'un des problèmes classiques des mécanismes d'équilibrage naïfs [17].

```
toutes les 60 secondes :  
  récupérer métriques moyennes (5 min) depuis InfluxDB  
  pour chaque noeud :  
    si CPU_moy > 80% ou RAM_moy > 85% :  
      identifier VM candidate (plus lourde, pas migrée récemment)  
      identifier noeud destination (CPU_moy < 30% et RAM_moy < 30%)  
      si noeud destination trouvé :  
        déclencher live migration via API Proxmox (VLAN 50)  
        enregistrer migration dans InfluxDB  
        appliquer délai anti-oscillation (10 min)
```

3.7.4. Stack de Monitoring

La stack de monitoring est déployée sur une VM dédiée au cluster, accessible uniquement via le VLAN 10 de management. Elle comprend InfluxDB pour le stockage des métriques et Grafana pour la visualisation. Proxmox VE est configuré pour pousser ses métriques nativement vers InfluxDB sans nécessiter d'agent externe [15], ce qui simplifie considérablement le déploiement et la maintenance de la stack.

3.8. Application de Démonstration : Sona-Web

3.8.1. Architecture Applicative

Sona-Web est une application web deux tiers déployée sur le cluster pour démontrer les capacités de haute disponibilité et d'équilibrage de charge de l'infrastructure. Elle est constituée de quatre VMs interconnectées via le VLAN 60 de production :

- **HAProxy** : point d'entrée unique de l'application, exposé sur le VLAN 80 (DMZ). Il distribue les requêtes HTTP entrantes entre les deux serveurs web selon l'algorithme round-robin, assurant l'équilibrage de charge applicatif. HAProxy surveille en permanence la disponibilité des serveurs web et retire automatiquement du pool tout serveur qui ne répond plus [16][26].
- **Apache VM 1 et VM 2** : deux serveurs web identiques servant une page affichant leur identité (nom du serveur et adresse IP). Cette identification permet au jury de visualiser en temps réel l'alternance des requêtes entre les deux serveurs lors du rafraîchissement de la page, prouvant que l'équilibrage de charge fonctionne correctement.
- **PostgreSQL** : serveur de base de données dont le volume de données est stocké sur un volume Ceph RBD dédié. C'est cette VM qui constitue la cible principale du scénario de démonstration HA : son volume RBD étant accessible depuis n'importe quel nœud du cluster, elle peut être redémarrée sur un nœud survivant sans aucune perte de données après une panne simulée.

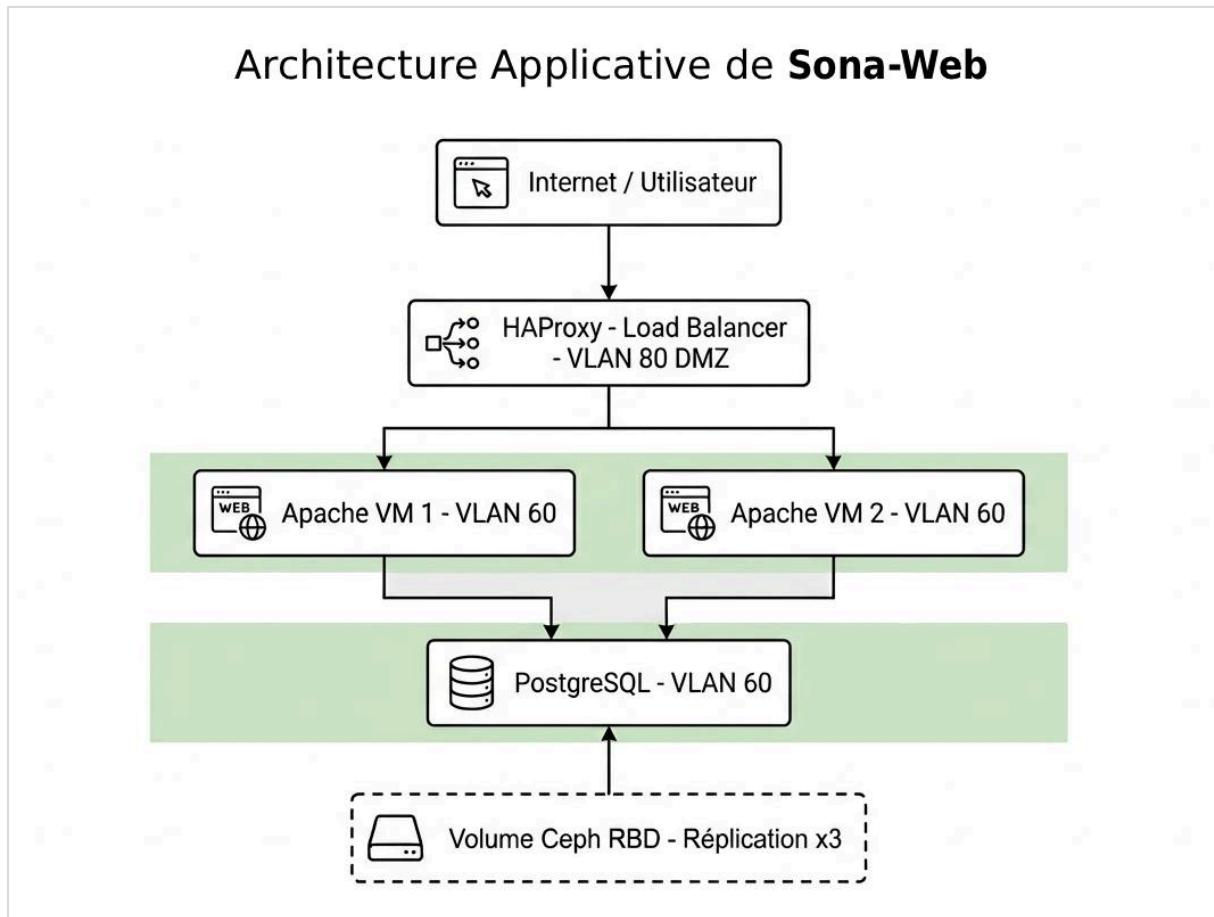


Figure 3-5: Architecture Applicative Sona-Web

3.8.2. Scénario de Validation HA

Le scénario de démonstration se déroule en quatre temps :

- **État normal** : nous accédons à Sona-Web via un navigateur et observons en rafraîchissant la page l'alternance entre le Serveur Web 1 et le Serveur Web 2, validant le fonctionnement du load balancing HAProxy.
- **Identification de la cible** : nous identifions sur quel noeud Proxmox la VM PostgreSQL est actuellement hébergée, par exemple PVE-01, visible depuis l'interface web Proxmox.

- **Simulation de panne** : nous éteignons brutalement le noeud PVE-01 depuis l'interface de l'hyperviseur hôte, simulant une panne matérielle complète.
- **Récupération automatique** : Corosync détecte la disparition du noeud en quelques secondes. Le HA Manager déclenche le redémarrage automatique de la VM PostgreSQL sur PVE-02 ou PVE-03. Sona-Web revient en ligne après 60 à 120 secondes [6], sans intervention humaine et sans perte de données grâce à la réplication Ceph.

3.9. Conclusion

Ce chapitre présente la conception complète de l'infrastructure Cloud privée hyperconvergente proposée. Nous avons détaillé chaque couche de l'architecture : le serveur physique hôte, les trois noeuds Proxmox virtuels formant le cluster, l'architecture réseau à neuf VLANs OVS, le dimensionnement du pool de stockage Ceph, le mécanisme DRS que nous développons, et l'application de démonstration Sona-Web. L'ensemble de ces choix de conception constitue une base cohérente et documentée pour la phase d'implémentation présentée dans le chapitre suivant.

Chapitre 4 : Implémentation et Déploiement

Bibliographie

- [1] I. N. Sonatrach. (2012, août 13). « *Historique et évolution* ». Institut-numérique. <https://www.institut-numerique.org/2-historique-et-evolution-5028f58cb09d3>
- [2] Schneider, J., Susnjara, S., et Smalley, I. (2024, 12 février). « *Hyperconverged infrastructure explained* ». IBM. <https://www.ibm.com/think/topics/hyperconverged-infrastructure>
- [3] MacAlpine, C. (2025, 27 mai). « *The top 10 hyperconverged infrastructure (HCI) software solutions* ». Expert Insights. <https://expertinsights.com/it-infrastructure/top-hyperconverged-infrastructure-hci-software>
- [4] Pande, A. (2025, February 23). « *Proxmox vs. Harvester: Can the enterprise-grade platform beat the community favorite?* » XDA Developers. <https://www.xda-developers.com/proxmox-vs-harvester/>
- [5] SUSE. (s. d.). « *Harvester HCI documentation* ». Harvester HCI. <https://harvesterhci.io/>
- [6] Proxmox Server Solutions. (s. d.). « *Proxmox virtual environment overview* ». Proxmox. <https://www.proxmox.com/en/products/proxmox-virtual-environment/overview>
- [7] Sahin, S. (2025, 2 octobre). « *Proxmox NAT configuration* ». ITTechNetworkK. <https://ittechnetworkk.github.io/posts/proxmoxnatconfiguration/proxmox-nat-configuration/>
- [8] Open vSwitch Authors. (s. d.). « *What is Open vSwitch ?* ». Documentation Open vSwitch. <https://docs.openvswitch.org/en/latest/intro/what-is-ovs/>
- [9] Open vSwitch Authors. (s. d.). « *Features* ». Open vSwitch. <https://www.openvswitch.org/features/>
- [10] Proxmox Server Solutions. (2025, 19 novembre). « *Deploy hyper-converged Ceph cluster* ». Proxmox VE Wiki. https://pve.proxmox.com/wiki/Deploy_Hyper-Converged_Ceph_Cluster
- [11] Proxmox Server Solutions. (2025, 18 décembre). « *Full mesh network for Ceph server* ». Proxmox VE Wiki. https://pve.proxmox.com/wiki/Full_Mesh_Network_for_Ceph_Server
- [12] Ceph Authors. (s. d.). « *Network configuration reference* ». Documentation Ceph. <https://docs.ceph.com/en/latest/rados/configuration/network-config-ref/>
- [13] Red Hat. (s. d.). « *Chapter 1 : The Ceph architecture* ». Red Hat Ceph Storage 4 Architecture Guide. https://docs.redhat.com/en/documentation/red_hat_ceph_storage/4/html/architecture_guide/the-ceph-architecture_arch
- [14] Chandak, A., Jaju, K., Kanfade, A., Lohiya, P., et Joshi, A. (2012, mai). « *Dynamic load balancing of virtual machines using QEMU-KVM* ». International Journal of Computer

- Applications, 46(6), 10-14.
<https://research.ijcaonline.org/volume46/number6/pxc3879263.pdf>
- [15] Proxmox Server Solutions. (2025, 6 août). « *Proxmox Backup Server* ». Proxmox Backup Server Wiki. https://pbs.proxmox.com/wiki/Main_Page
 - [16] Mr.PlanB. (2025, 9 septembre). « *Goodbye DRS, hello PBS : What I wish I knew before ditching VMware for Proxmox* ». Medium.
<https://medium.com/@PlanB./goodbye-drs-hello-pbs-what-i-wish-i-knew-before-ditching-vmware-for-proxmox-26f9dec3e9f5>
 - [17] Watts, J. (1995). « *A practical approach to dynamic load balancing* » [Mémoire de master, California Institute of Technology]. Caltech THESIS.
https://thesis.caltech.edu/6924/1/Watts_j_1995.pdf
 - [18] Broadcom. (s. d.). « *VMware DRS overview : Optimizing resource allocation in vSphere cluster* ». Broadcom Knowledge Base.
<https://knowledge.broadcom.com/external/article/391137/vmware-drs-overview-optimizing-resource.html>
 - [19] KVM Authors. (s. d.). « *Nested guests* ». Linux KVM Wiki.
https://www.linux-kvm.org/page/Nested_Guests
 - [20] Corosync Project. (s. d.). « *The Corosync cluster engine* ». Corosync.
<https://corosync.github.io/corosync/>
 - [21] Red Hat. (s. d.). « *Configuring and managing high availability clusters* ». Red Hat Enterprise Linux 9 Documentation.
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_high_availability_clusters/index
 - [22] Broadcom. (s. d.). « *vSphere availability* ». VMware vSphere 8.0 Documentation.
<https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0/vsphere-availability.html>
 - [23] VMware. (s. d.). « *vSphere DRS performance* ». VMware Documentation.
<https://www.vmware.com/docs/vsphere6-drs-perf>
 - [24] InfluxData. (s. d.). « *InfluxDB v2 documentation* ». InfluxData.
<https://docs.influxdata.com/influxdb/v2/>
 - [25] Grafana Labs. (s. d.). « *Grafana documentation* ». Grafana Labs.
<https://grafana.com/docs/grafana/latest/>
 - [26] Willy Tarreau. (s. d.). « *HAProxy version 2.8 : configuration manuelle* ». HAProxy.
<https://www.haproxy.org/download/2.8/doc/configuration.txt>
 - [27] Weil, S. A., Brandt, S. A., Miller, E. L., & Long, D. D. (2006). Ceph: A scalable,

- high-performance distributed file system. In Proceedings of the 7th symposium on Operating systems design and implementation (OSDI '06). USENIX Association
- [28] Weil, S. A., Brandt, S. A., Miller, E. L., & Maltzahn, C. (2007). CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data. In Proceedings of the 2007 ACM/IEEE conference on Supercomputing (SC '07). ACM.
- [29] Ceph Documentation. (2024). RADOS Block Device (RBD).
<https://docs.ceph.com/en/latest/rbd/>
- [30] https://docs.redhat.com/en/documentation/red_hat_ceph_storage/4/html/architecture_guide/the-ceph-architecture_arch
- [31] <https://docs.ceph.com/en/latest/rados/configuration/network-config-ref/>
- [32] Van der Ster, D., & Wiebalck, A. (2025). Ceph at CERN in the multi-datacentre era. EPJ Web of Conferences, 337, 01331. <https://doi.org/10.1051/epjconf/202533701331>
- [33] Proxmox Server Solutions. (2024). Proxmox VE Ceph Server.
https://pve.proxmox.com/wiki/Ceph_Server
- [34]
- [35]
- [36]
- [37]
- [38]
- [39]
- [40]
- [41]
- [42]